



 THE TERMINAL FIELD GUIDE

Claude Code

Command & Flag Cheatsheet



Every in-session slash command and every `claude` launch flag — what it means, how to use it, an example, and when to reach for it.

90

SLASH COMMANDS

60

CLI FLAGS

24

CLI COMMANDS

12

CATEGORIES

Legend:

SKILL bundled prompt Claude can also auto-invoke

WORKFLOW fans work across background subagents

SANDBOX ONLY never on a machine with real credentials

<arg> required **[arg]** optional

In-session slash commands

Type `/` at the start of a message to control Claude Code mid-session. Text after the name is passed as arguments.

How they work: a command is only recognised at the start of your message. Not every command appears for every user — availability depends on your platform, plan, and environment (e.g. `/desktop` needs macOS/Windows + a subscription; `/upgrade` shows only on Pro and Max). Connected MCP servers add commands in the form `/mcp__<server>__<prompt>`.

• Session & context

13

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/clear [name]</code>	Start a fresh conversation with empty context; the old one stays in <code>/resume</code> . Aliases: <code>/reset</code> , <code>/new</code> .	<code>/clear</code>	Begin an unrelated task while keeping project memory.
<code>/compact [instructions]</code>	Summarise the conversation so far to free up context. Optional focus instructions.	<code>/compact focus on the auth work</code>	The conversation is long and the context window is filling up.
<code>/context [all]</code>	Visualise context usage as a coloured grid with optimisation hints. 'all' expands the breakdown.	<code>/context all</code>	Find out what is eating your context window.
<code>/rewind</code>	Roll conversation and/or code back to a checkpoint, or summarise from a chosen message. Aliases: <code>/checkpoint</code> , <code>/undo</code> .	<code>/rewind</code>	Undo a bad edit or abandon an experiment.
<code>/resume [session]</code>	Resume a conversation by ID or name, or open the picker. Alias: <code>/continue</code> .	<code>/resume auth-refactor</code>	Return to earlier work, including background sessions.
<code>/branch [name]</code>	Branch the conversation at this point; switches you in and preserves the original.	<code>/branch try-redis</code>	Explore a different direction without losing current state.
<code>/fork <directive></code>	Spawn a forked subagent that inherits the full conversation and works while you keep going.	<code>/fork write tests for the parser</code>	Hand a side task to a background agent.
<code>/btw <question></code>	Ask a quick side question that is not added to the main conversation history.	<code>/btw what does git rebase do</code>	A quick aside that should not bloat history.
<code>/goal [condition clear]</code>	Set a goal; Claude keeps working across turns until the condition is met.	<code>/goal all tests pass</code>	A persistent objective spanning many turns.

● Session & context

13

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/rename [name]</code>	Rename the session; shown on the prompt bar. Auto-generates a name if omitted.	<code>/rename payments-feature</code>	Label a session so it is easy to find in <code>/resume</code> .
<code>/recap</code>	Generate a one-line summary of the current session on demand.	<code>/recap</code>	Quick reminder of where you left off.
<code>/export [filename]</code>	Export the current conversation as plain text to a file or the clipboard.	<code>/export session.txt</code>	Save or share a transcript.
<code>/copy [N]</code>	Copy the last assistant response (or the Nth-latest) to the clipboard.	<code>/copy 2</code>	Grab Claude's output or a specific code block.

● Models & reasoning

5

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/model [model]</code>	Switch the model and save it as the default for new sessions. Press 's' for session-only.	<code>/model opus</code>	Move to a stronger or cheaper model mid-task.
<code>/effort [level auto]</code>	Set effort: low / medium / high / xhigh / max / ultracode. Available levels depend on the model.	<code>/effort high</code>	Dial reasoning up for a hard problem, down to save cost.
<code>/fast [on off]</code>	Toggle fast mode for low-latency responses.	<code>/fast on</code>	Quick answers when deep reasoning is not needed.
<code>/plan [description]</code>	Enter plan mode from the prompt; optionally start with a task immediately.	<code>/plan fix the auth bug</code>	Plan a large change before any edits are made.
<code>/ultraplan <prompt></code>	Draft a plan in a cloud session, review it in the browser, then execute remotely or locally.	<code>/ultraplan design a caching layer</code>	Heavy planning you want to review on the web.

● Review & ship

6

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/diff</code>	Open an interactive diff viewer: uncommitted changes plus per-turn diffs.	<code>/diff</code>	Review what changed before committing.

● Review & ship

6

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/code-review [level] [--fix] [--comment] [target]</code> SKILL	Review the diff for correctness bugs and cleanups. --fix applies findings; 'ultra' runs a deep cloud review.	<code>/code-review high --fix</code>	A quality pass before opening a PR.
<code>/review [PR]</code>	Review a pull request locally in your current session (read-only).	<code>/review 1234</code>	A focused PR review without leaving the terminal.
<code>/security-review</code>	Analyse pending changes on the branch for security vulnerabilities (injection, auth, data exposure).	<code>/security-review</code>	Catch security risks before shipping.
<code>/simplify [target]</code> SKILL	Cleanup-only review (reuse, simplification, efficiency, abstraction) that applies the fixes; does not hunt bugs.	<code>/simplify src/</code>	Tidy code without bug-hunting.
<code>/ultrareview [PR]</code>	Deep multi-agent code review in a cloud sandbox. Preferred form: <code>/code-review ultra</code> .	<code>/ultrareview 1234</code>	Thorough review of a large or risky change.

● Parallel & background work

6

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/agents</code>	Manage subagent configurations Claude can delegate side tasks to.	<code>/agents</code>	Set up specialised agents for a project.
<code>/tasks</code>	View and manage everything running in the background. Also: <code>/bashes</code> .	<code>/tasks</code>	Check on background jobs.
<code>/background [prompt]</code>	Detach the session to run as a background agent and free the terminal. Alias: <code>/bg</code> .	<code>/background keep running the migration</code>	A long task you do not want to babysit.
<code>/batch <instruction></code> SKILL	Decompose a large change into 5-30 units, run each in its own git worktree, and open a PR per unit.	<code>/batch migrate src/ from Solid to React</code>	Codebase-wide changes done in parallel.
<code>/workflows</code>	Open the workflow progress view to watch, pause, resume, or save runs.	<code>/workflows</code>	Monitor long-running background workflows.

• Parallel & background work

6

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/stop</code>	Stop the current background session (only while attached); the transcript and worktree are kept.	<code>/stop</code>	Halt a background agent you are attached to.

• Setup & project

11

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/init</code>	Initialise the project with a starter CLAUDE.md guide. Set CLAUDE_CODE_NEW_INIT=1 for an interactive flow.	<code>/init</code>	First session in a new repository.
<code>/memory</code>	Edit CLAUDE.md memory files and manage auto-memory entries.	<code>/memory</code>	Refine what Claude remembers about the project.
<code>/mcp</code>	Manage MCP server connections and OAuth authentication.	<code>/mcp</code>	Connect external tools and data sources.
<code>/permissions</code>	Manage allow / ask / deny rules for tool permissions. Alias: <code>/allowed-tools</code> .	<code>/permissions</code>	Set the approval rules you want.
<code>/hooks</code>	View hook configurations for tool lifecycle events.	<code>/hooks</code>	Inspect automation around tool events.
<code>/skills</code>	List available skills; sort by token count, or hide a skill from Claude or the menu.	<code>/skills</code>	See and manage installed skills.
<code>/add-dir <path></code>	Add a working directory for file access during this session.	<code>/add-dir ../shared-lib</code>	Work across multiple repos or folders.
<code>/plugin [subcommand]</code>	Manage plugins. Subcommands include list, install, enable, disable.	<code>/plugin install code-review</code>	Extend Claude Code with packaged tooling.
<code>/reload-plugins [--force]</code>	Reload active plugins to apply pending changes without restarting.	<code>/reload-plugins</code>	Apply plugin edits mid-session.
<code>/reload-skills</code>	Re-scan skill and command directories so newly added ones become available.	<code>/reload-skills</code>	Pick up a skill you just created on disk.

• Setup & project

11

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/fewer-permission-prompts</code> SKILL	Scan transcripts for common read-only calls and add an allowlist to project settings.	<code>/fewer-permission-prompts</code>	Cut down on repetitive permission prompts.

• Appearance & config

9

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/config</code>	Open Settings to adjust theme, model, output style, and preferences. Alias: <code>/settings</code> .	<code>/config</code>	Tune Claude Code to your workflow.
<code>/theme</code>	Change the colour theme: auto, light, dark, colourblind-accessible, ANSI, or custom.	<code>/theme</code>	Match your terminal's look and feel.
<code>/color [color default]</code>	Set the prompt-bar colour for this session (red, blue, green, yellow, purple, orange, pink, cyan).	<code>/color blue</code>	Tell several open sessions apart.
<code>/statusline</code>	Configure the status line; describe what you want or auto-configure from your shell prompt.	<code>/statusline</code>	Customise the info bar at the bottom.
<code>/keybindings</code>	Open your keyboard shortcuts file.	<code>/keybindings</code>	Customise key bindings.
<code>/terminal-setup</code>	Configure terminal keybindings for Shift+Enter and similar shortcuts.	<code>/terminal-setup</code>	Fix newline keys in VS Code, Cursor, or Zed.
<code>/tui [default fullscreen]</code>	Set the terminal UI renderer and relaunch with the conversation intact.	<code>/tui fullscreen</code>	Enable the flicker-free fullscreen renderer.
<code>/focus</code>	Toggle focus view: last prompt, a one-line tool summary, and the final response only.	<code>/focus</code>	A cleaner, distraction-free reading view.
<code>/scroll-speed</code>	Adjust mouse-wheel scroll speed interactively (fullscreen only).	<code>/scroll-speed</code>	Tune scrolling in the fullscreen renderer.

• Account, plan & usage

7

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/login</code>	Sign in to your Anthropic account.	<code>/login</code>	Authenticate before working.
<code>/logout</code>	Sign out from your Anthropic account.	<code>/logout</code>	Switch accounts or end access.
<code>/usage</code>	Show session cost, plan usage limits, and activity stats. Aliases: <code>/cost</code> , <code>/stats</code> .	<code>/usage</code>	Track spend and how close you are to limits.
<code>/usage-credits</code>	Configure usage credits so you can keep working after hitting a limit (was <code>/extra-usage</code>).	<code>/usage-credits</code>	Avoid being blocked when you hit a cap.
<code>/upgrade</code>	Open the upgrade page to move to a higher plan tier (Pro and Max only).	<code>/upgrade</code>	Need higher limits or more features.
<code>/privacy-settings</code>	View and update your privacy settings (Pro and Max subscribers).	<code>/privacy-settings</code>	Control data and privacy preferences.
<code>/passes</code>	Share a free week of Claude Code with friends (only if your account is eligible).	<code>/passes</code>	Invite a colleague to try Claude Code.

• Diagnostics & help

9

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/help</code>	Show help and the full list of available commands.	<code>/help</code>	Orient yourself or discover commands.
<code>/doctor</code>	Diagnose and verify your installation and settings; press 'f' to have Claude fix issues.	<code>/doctor</code>	Something is misconfigured or broken.
<code>/debug [description]</code> SKILL	Enable debug logging for the session and troubleshoot by reading the debug log.	<code>/debug MCP server won't connect</code>	Investigate a runtime problem.
<code>/status</code>	Open Settings (Status tab): version, model, account, and connectivity. Works mid-response.	<code>/status</code>	Quick health and account check.
<code>/release-notes</code>	View the changelog in an interactive version picker.	<code>/release-notes</code>	See what changed in a given version.

• Diagnostics & help

9

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/feedback [report]</code>	Submit feedback, report a bug, or share your conversation. Aliases: <code>/bug</code> , <code>/share</code> .	<code>/feedback</code>	Report a problem with session context attached.
<code>/insights</code>	Generate a report analysing your sessions: project areas, patterns, and friction points.	<code>/insights</code>	Reflect on how you use Claude Code.
<code>/heapdump</code>	Write a JS heap snapshot and memory breakdown to your Desktop for diagnosing memory use.	<code>/heapdump</code>	Diagnose high memory usage.
<code>/powerup</code>	Learn Claude Code features through quick interactive lessons with animated demos.	<code>/powerup</code>	Discover features you have not tried.

• Connect & continue elsewhere

11

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/desktop</code>	Continue the session in the Claude Code Desktop app (macOS / Windows). Alias: <code>/app</code> .	<code>/desktop</code>	Move from terminal to the desktop app.
<code>/mobile</code>	Show a QR code to download the Claude mobile app. Aliases: <code>/ios</code> , <code>/android</code> .	<code>/mobile</code>	Continue on your phone.
<code>/remote-control</code>	Make this session controllable from claude.ai or the Claude app. Alias: <code>/rc</code> .	<code>/remote-control</code>	Drive a local session from another device.
<code>/teleport</code>	Pull a Claude Code on the web session into this terminal. Alias: <code>/tp</code> .	<code>/teleport</code>	Bring cloud work down to your machine.
<code>/remote-env</code>	Choose the default environment for cloud agents.	<code>/remote-env</code>	Configure where web sessions run.
<code>/autofix-pr [prompt]</code>	Spawn a web session that watches the branch's PR and pushes fixes when CI fails or reviewers comment.	<code>/autofix-pr only fix lint errors</code>	Keep a PR green automatically.
<code>/chrome</code>	Configure Claude in Chrome settings for web automation and testing.	<code>/chrome</code>	Set up browser automation.

• Connect & continue elsewhere

11

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/ide</code>	Manage IDE integrations and show connection status.	<code>/ide</code>	Connect or check your editor integration.
<code>/install-github-app</code>	Set up the Claude GitHub Actions app for a repository.	<code>/install-github-app</code>	Enable Claude on GitHub PRs and CI.
<code>/install-slack-app</code>	Install the Claude Slack app via OAuth.	<code>/install-slack-app</code>	Bring Claude into Slack.
<code>/web-setup</code>	Connect your GitHub account to Claude Code on the web using your local gh CLI credentials.	<code>/web-setup</code>	Link GitHub for cloud sessions.

• More bundled skills & workflows

8

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/run</code> SKILL	Launch and drive your app to see a change working in the running app, not just in tests.	<code>/run</code>	Confirm a change behaves against the live app.
<code>/verify</code> SKILL	Build and run your app to confirm a change does what it should, without falling back to tests.	<code>/verify</code>	Validate a change end-to-end.
<code>/run-skill-generator</code> SKILL	Teach <code>/run</code> and <code>/verify</code> how to build, launch, and drive your project from a clean environment.	<code>/run-skill-generator</code>	One-time setup so <code>/run</code> and <code>/verify</code> work.
<code>/claude-api [migrate managed-agents-onboard]</code> SKILL	Load Claude API reference for your language; migrate code to a new model or onboard a Managed Agent.	<code>/claude-api migrate</code>	Build on or upgrade Claude API code.
<code>/loop [interval] [prompt]</code> SKILL	Run a prompt repeatedly while the session stays open. Alias: <code>/proactive</code> .	<code>/loop 5m check if the deploy finished</code>	Poll for a condition on a schedule.
<code>/deep-research <question></code> WORKFLOW	Fan out web searches, fetch and cross-check sources, and synthesise a cited report.	<code>/deep-research compare vector databases</code>	Need a thorough, sourced answer.
<code>/schedule [description]</code>	Create, update, list, or run routines that execute on cloud infrastructure. Alias: <code>/routines</code> .	<code>/schedule daily dependency check</code>	Automate recurring tasks.

• More bundled skills & workflows

8

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/team-onboarding</code>	Generate a teammate ramp-up guide from your project's local Claude Code usage.	<code>/team-onboarding</code>	Help a new developer get productive fast.

• Sandbox & extras

5

COMMAND · USAGE	WHAT IT DOES	EXAMPLE	WHEN TO USE IT
<code>/sandbox</code>	Toggle sandbox mode (supported platforms only).	<code>/sandbox</code>	Run with extra isolation.
<code>/voice [hold tap off]</code>	Toggle voice dictation, or enable a specific mode. Requires a Claude.ai account.	<code>/voice tap</code>	Dictate instead of typing.
<code>/radio</code>	Open Claude FM lo-fi radio in your browser; prints the stream URL if no browser is available.	<code>/radio</code>	Background music while you work.
<code>/stickers</code>	Order Claude Code stickers.	<code>/stickers</code>	Get some swag.
<code>/exit</code>	Exit the CLI; in an attached background session this detaches and the session keeps running. Alias: <code>/quit</code> .	<code>/exit</code>	Leave the session.

2A · Launch commands & subcommands

COMMAND	WHAT IT DOES	EXAMPLE
<code>claude</code>	Start an interactive session.	<code>claude</code>
<code>claude "query"</code>	Start an interactive session with an initial prompt.	<code>claude "explain this project"</code>
<code>claude -p "query"</code>	One-shot query via the SDK, then exit (the workhorse for scripting/CI).	<code>claude -p "explain this function"</code>
<code>cat file claude -p "query"</code>	Process piped content in one-shot mode.	<code>cat logs.txt claude -p "explain"</code>
<code>claude -c</code>	Continue the most recent conversation in the current directory.	<code>claude -c</code>
<code>claude -c -p "query"</code>	Continue the most recent conversation via the SDK.	<code>claude -c -p "check for type errors"</code>
<code>claude -r "<session>" "query"</code>	Resume a session by ID or name with a new prompt.	<code>claude -r "auth-refactor" "finish this PR"</code>
<code>claude update</code>	Update Claude Code to the latest version.	<code>claude update</code>
<code>claude install [version]</code>	Install or reinstall the native binary (a version, or 'stable' / 'latest').	<code>claude install stable</code>
<code>claude auth login logout status</code>	Sign in, sign out, or print auth status. <code>--console</code> signs in for API billing.	<code>claude auth login --console</code>
<code>claude agents</code>	Open the agent view to monitor and dispatch parallel background sessions.	<code>claude agents --json</code>
<code>claude attach <id></code>	Attach to a background session in this terminal.	<code>claude attach 7c5dcf5d</code>
<code>claude logs <id></code>	Print recent output from a background session.	<code>claude logs 7c5dcf5d</code>
<code>claude respawn <id></code>	Restart a background session (running or stopped) with its conversation intact.	<code>claude respawn 7c5dcf5d</code>
<code>claude stop <id></code>	Stop a background session. Also accepts <code>claude kill</code> .	<code>claude stop 7c5dcf5d</code>

COMMAND	WHAT IT DOES	EXAMPLE
<code>claude rm <id></code>	Remove a background session from the list (the transcript stays on disk).	<code>claude rm 7c5dcf5d</code>
<code>claude mcp</code>	Configure Model Context Protocol (MCP) servers.	<code>claude mcp list</code>
<code>claude plugin</code>	Manage plugins. Alias: <code>claude plugins</code> .	<code>claude plugin install code-review@official</code>
<code>claude project purge [path]</code>	Delete all local Claude Code state for a project. <code>--dry-run</code> previews.	<code>claude project purge ~/work/repo --dry-run</code>
<code>claude remote-control</code>	Start a Remote Control server (no local interactive session).	<code>claude remote-control --name "My Project"</code>
<code>claude setup-token</code>	Generate a long-lived OAuth token for CI and scripts (requires a subscription).	<code>claude setup-token</code>
<code>claude ultrareview [target]</code>	Run ultrareview non-interactively; exits 0 on success, 1 on failure.	<code>claude ultrareview 1234 --json</code>
<code>claude auto-mode defaults</code>	Print the built-in auto-mode classifier rules as JSON.	<code>claude auto-mode defaults > rules.json</code>
<code>claude daemon status stop</code>	Inspect or stop the background-session supervisor process.	<code>claude daemon stop --any --keep-workers</code>

2B · CLI flags & arguments

● Print mode & scripting			14
FLAG	WHAT IT DOES	EXAMPLE	
<code>--print, -p</code>	Print the response without interactive mode (one-shot, SDK-backed).	<code>claude -p "query"</code>	
<code>--output-format</code>	Output format for print mode: text, json, or stream-json.	<code>claude -p "query" --output-format json</code>	
<code>--input-format</code>	Input format for print mode: text or stream-json.	<code>--input-format stream-json</code>	
<code>--include-partial-messages</code>	Emit token-level stream events (needs <code>--print</code> and stream-json).	<code>--include-partial-messages</code>	
<code>--include-hook-events</code>	Include hook lifecycle events in the output stream (needs stream-json).	<code>--include-hook-events</code>	

● Print mode & scripting

14

FLAG	WHAT IT DOES	EXAMPLE
<code>--replay-user-messages</code>	Re-emit stdin user messages back on stdout for acknowledgement.	<code>--replay-user-messages</code>
<code>--prompt-suggestions</code>	Emit a predicted next-prompt after each turn (needs print + stream-json + verbose).	<code>--prompt-suggestions</code>
<code>--json-schema</code>	Return validated JSON matching a JSON Schema after the workflow completes.	<code>--json-schema '{"type":"object"}'</code>
<code>--max-turns</code>	Limit the number of agentic turns; errors when the limit is hit (print mode).	<code>claude -p --max-turns 3 "query"</code>
<code>--max-budget-usd</code>	Maximum dollars to spend on API calls before stopping (print mode).	<code>claude -p --max-budget-usd 5.00 "query"</code>
<code>--bare</code>	Minimal mode: skip discovery of hooks, skills, plugins, MCP, memory, CLAUDE.md for fast scripts.	<code>claude --bare -p "query"</code>
<code>--exclude-dynamic-system-prompt-sections</code>	Move per-machine prompt sections into the first user message for better cache reuse.	<code>claude -p --exclude-dynamic-system-prompt-sections "query"</code>
<code>--no-session-persistence</code>	Do not save the session to disk; it cannot be resumed (print mode).	<code>claude -p --no-session-persistence "query"</code>
<code>--init / --init-only / --maintenance</code>	Run Setup / SessionStart hooks before (or instead of) a session.	<code>claude --init-only</code>

● Sessions

10

FLAG	WHAT IT DOES	EXAMPLE
<code>--continue, -c</code>	Load the most recent conversation in the current directory.	<code>claude --continue</code>
<code>--resume, -r</code>	Resume a session by ID or name, or open an interactive picker.	<code>claude --resume auth-refactor</code>
<code>--fork-session</code>	When resuming, create a new session ID instead of reusing the original.	<code>claude --resume abc123 --fork-session</code>

• Sessions

10

FLAG	WHAT IT DOES	EXAMPLE
<code>--from-pr</code>	Resume sessions linked to a specific pull request (number or URL).	<code>claude --from-pr 123</code>
<code>--session-id</code>	Use a specific session ID (must be a valid UUID).	<code>claude --session-id 550e8400-...</code>
<code>--name, -n</code>	Set a display name for the session, shown in <code>/resume</code> and the title.	<code>claude -n "my-feature-work"</code>
<code>--remote</code>	Create a new web session on claude.ai with the given task.	<code>claude --remote "fix the login bug"</code>
<code>--teleport</code>	Resume a web session in your local terminal.	<code>claude --teleport</code>
<code>--bg</code>	Start the session as a background agent and return immediately.	<code>claude --bg "investigate the flaky test"</code>
<code>--exec</code>	Run a shell command as a PTY-backed background job (use with <code>--bg</code>).	<code>claude --bg --exec 'pytest -x'</code>

• Model & agents

5

FLAG	WHAT IT DOES	EXAMPLE
<code>--model</code>	Set the model: an alias (sonnet, opus) or a full model name.	<code>claude --model claude-sonnet-4-6</code>
<code>--fallback-model</code>	Fall back to another model when the default is overloaded (print / background).	<code>claude -p --fallback-model sonnet "query"</code>
<code>--effort</code>	Set the effort level for the session: low, medium, high, xhigh, max.	<code>claude --effort high</code>
<code>--agent</code>	Use a specific agent for the session (overrides the agent setting).	<code>claude --agent my-custom-agent</code>
<code>--agents</code>	Define custom subagents dynamically via JSON.	<code>claude --agents '{"reviewer":{...}}'</code>

• Permissions & tools

8

FLAG	WHAT IT DOES	EXAMPLE
<code>--permission-mode</code>	Start in default, acceptEdits, plan, auto, dontAsk, or bypassPermissions.	<code>claude --permission-mode plan</code>
<code>--dangerously-skip-permissions</code> SANDBOX ONLY	Skip all permission prompts (= bypassPermissions). Sandboxed environments only.	<code>claude --dangerously-skip-permissions</code>
<code>--allow-dangerously-skip-permissions</code> SANDBOX ONLY	Add bypassPermissions to the Shift+Tab cycle without starting in it.	<code>claude --permission-mode plan --allow-dangerously-skip-permissions</code>
<code>--allowedTools</code>	Tools that run without prompting for permission.	<code>--allowedTools "Bash(git log *)" "Read"</code>
<code>--disallowedTools</code>	Deny rules; a bare tool name removes it from the model's context.	<code>--disallowedTools "Bash(rm *)" "Edit"</code>
<code>--tools</code>	Restrict which built-in tools are available ("" = none, "default" = all).	<code>claude --tools "Bash,Edit,Read"</code>
<code>--permission-prompt-tool</code>	MCP tool that handles permission prompts in non-interactive mode.	<code>--permission-prompt-tool mcp_auth_tool</code>
<code>--disable-slash-commands</code>	Disable all skills and commands for this session.	<code>claude --disable-slash-commands</code>

• System prompt

4

FLAG	WHAT IT DOES	EXAMPLE
<code>--system-prompt</code>	Replace the entire system prompt with custom text.	<code>claude --system-prompt "You are a Python expert"</code>
<code>--system-prompt-file</code>	Replace the system prompt with the contents of a file.	<code>claude --system-prompt-file ./prompt.txt</code>
<code>--append-system-prompt</code>	Append custom text to the end of the default system prompt.	<code>claude --append-system-prompt "Always use TypeScript"</code>
<code>--append-system-prompt-file</code>	Append the contents of a file to the default prompt.	<code>claude --append-system-prompt-file ./rules.txt</code>

• Directories, settings & MCP

7

FLAG	WHAT IT DOES	EXAMPLE
<code>--add-dir</code>	Add working directories for Claude to read and edit files.	<code>claude --add-dir ../apps ../lib</code>
<code>--settings</code>	Path to a settings JSON file or inline JSON; overrides matching keys for the session.	<code>claude --settings ./settings.json</code>
<code>--setting-sources</code>	Which setting sources to load: user, project, local.	<code>claude --setting-sources user,project</code>
<code>--mcp-config</code>	Load MCP servers from JSON files or strings (space-separated).	<code>claude --mcp-config ./mcp.json</code>
<code>--strict-mcp-config</code>	Use only MCP servers from --mcp-config, ignoring all others.	<code>claude --strict-mcp-config --mcp-config ./mcp.json</code>
<code>--plugin-dir</code>	Load a plugin from a directory or .zip for this session only.	<code>claude --plugin-dir ./my-plugin</code>
<code>--plugin-url</code>	Fetch a plugin .zip from a URL for this session only.	<code>claude --plugin-url https://example.com/plugin.zip</code>

• Worktrees, IDE & integrations

7

FLAG	WHAT IT DOES	EXAMPLE
<code>--worktree, -w</code>	Start in an isolated git worktree; pass #<number> or a PR URL to branch from it.	<code>claude -w feature-auth</code>
<code>--tmux</code>	Create a tmux session for the worktree (requires --worktree).	<code>claude -w feature-auth --tmux</code>
<code>--ide</code>	Automatically connect to your IDE on startup if exactly one is available.	<code>claude --ide</code>
<code>--chrome / --no-chrome</code>	Enable or disable Chrome browser integration for this session.	<code>claude --chrome</code>
<code>--remote-control, --rc</code>	Start an interactive session that can also be controlled from claude.ai.	<code>claude --remote-control "My Project"</code>
<code>--teammate-mode</code>	Agent-team teammate display: auto, in-process, or tmux.	<code>claude --teammate-mode in-process</code>
<code>--channels</code>	MCP servers whose channel notifications to listen for (research preview).	<code>claude --channels plugin:my-notifier@mkt</code>

FLAG	WHAT IT DOES	EXAMPLE
<code>--debug</code>	Enable debug mode with optional category filtering.	<code>claude --debug "api,mcp"</code>
<code>--debug-file <path></code>	Write debug logs to a specific file (implicitly enables debug mode).	<code>claude --debug-file /tmp/claude-debug.log</code>
<code>--verbose</code>	Verbose logging; shows full turn-by-turn output.	<code>claude --verbose</code>
<code>--betas</code>	Beta headers to include in API requests (API-key users only).	<code>claude --betas interleaved-thinking</code>
<code>--version, -v</code>	Output the version number.	<code>claude -v</code>

Scripting starter: `claude -p "review the diff and flag security concerns" --output-format json --max-turns 5 --permission-mode plan -- pair -p with --output-format json for structured output (cost, duration, turns), and prefer --permission-mode plan or a sandbox over --dangerously-skip-permissions on any machine with real credentials.`